

Image Optimization Guide

Automatic Image Processing

Your blog now automatically optimizes every uploaded image with: - **Thumbnail generation** (150x150, 400x400, 800x800) - **WebP conversion** (30-80% smaller files) - **Smart format selection** (JPEG for photos)

How It Works

Upload Flow

1. **User uploads image** → Base64 encoded
2. **Backend receives** → Decodes and validates
3. **Image processing** → Creates 5 variants:
 - Original (re-encoded as JPEG @ 85% quality)
 - Thumbnail (150x150 max)
 - Small (400x400 max)
 - Medium (800x800 max)
 - WebP (original size, 80% quality)
4. **All variants stored** → Turso database
5. **URLs returned** → For each variant

Automatic Optimizations

1. **Re-encoding:** - All images converted to JPEG (except WebP variant) - 85% quality (great balance of size/quality) - Removes metadata (EXIF, etc.)
 2. **Thumbnail Generation:** - Maintains aspect ratio - Uses Lanczos3 filter (highest quality) - Only creates if original is larger
 3. **WebP Conversion:** - 80% quality setting - 30-80% smaller than JPEG - Preserves dimensions
-

Image Variants

Original

- **Format:** JPEG
- **Quality:** 85%
- **Use:** High-quality display
- **URL:** /api/images/{id}

Thumbnail (150x150)

- **Max dimension:** 150px

- **Use:** List views, avatars
- **URL:** `/api/images/{id}?variant=thumbnail`

Small (400x400)

- **Max dimension:** 400px
- **Use:** Mobile views, cards
- **URL:** `/api/images/{id}?variant=small`

Medium (800x800)

- **Max dimension:** 800px
- **Use:** Desktop views
- **URL:** `/api/images/{id}?variant=medium`

WebP

- **Format:** WebP
- **Quality:** 80%
- **Use:** Modern browsers
- **URL:** `/api/images/{id}?variant=webp`

API Usage

Upload Response

```
{
  "success": true,
  "data": {
    "id": "uuid-here",
    "url": "http://localhost:3000/api/images/uuid-here",
    "filename": "photo.jpg",
    "variants": [
      {
        "variant": "original",
        "url": "...?variant=original",
        "width": 1920,
        "height": 1080,
        "size": 245678,
        "content_type": "image/jpeg"
      },
      {
        "variant": "thumbnail",
        "url": "...?variant=thumbnail",
        "width": 150,
```

```

        "height": 84,
        "size": 8456,
        "content_type": "image/jpeg"
    },
    {
        "variant": "webp",
        "url": "...?variant=webp",
        "width": 1920,
        "height": 1080,
        "size": 123456,
        "content_type": "image/webp"
    }
    // ... more variants
],
"optimization_info": "Original: 320 KB, WebP: 120 KB (62% smaller)"
}
}

```

Get Specific Variant

Get original

```
curl http://localhost:3000/api/images/{id}
```

Get thumbnail

```
curl http://localhost:3000/api/images/{id}?variant=thumbnail
```

Get WebP version

```
curl http://localhost:3000/api/images/{id}?variant=webp
```

Frontend Usage

Responsive Images

```

<picture>
  <!-- WebP for modern browsers -->
  <source
    srcset="/api/images/{id}?variant=webp"
    type="image/webp"
  />

  <!-- Responsive JPEG fallback -->
  <source
    media="(max-width: 400px)"
    srcset="/api/images/{id}?variant=small"
  />

```

```

<source
  media="(max-width: 800px)"
  srcset="/api/images/{id}?variant=medium"
/>

<!-- Default -->

</picture>

```

React Component

```

interface ResponsiveImageProps {
  imageId: string
  alt: string
  sizes?: string
}

function ResponsiveImage({ imageId, alt, sizes }: ResponsiveImageProps) {
  const base = `/api/images/${imageId}`

  return (
    <picture>
      <source srcSet={`${base}?variant=webp`} type="image/webp" />
      <source media="(max-width: 400px)" srcSet={`${base}?variant=small`} />
      <source media="(max-width: 800px)" srcSet={`${base}?variant=medium`} />
      <img
        src={base}
        alt={alt}
        loading="lazy"
        className="w-full h-auto"
      />
    </picture>
  )
}

```

Next.js Image Component

```

import Image from 'next/image'

<Image
  src={`/api/images/${imageId}?variant=medium`}
  alt="Post image"

```

```
width={800}  
height={600}  
quality={85}  
</>
```

Performance Benefits

File Size Reduction

Example 1920x1080 photo:

Variant	Size	vs Original
Original Upload	2.1 MB	-
Re-encoded JPEG	320 KB	-85%
Thumbnail	8 KB	-99.6%
Small	45 KB	-97.8%
Medium	120 KB	-94.3%
WebP	120 KB	-94.3%

Total saved: ~6 MB per image when using variants!

Loading Speed

On 3G connection (750 Kbps):

Variant	Load Time
Original (2.1 MB)	22s
Re-encoded (320 KB)	3.4s
Small (45 KB)	0.5s
WebP (120 KB)	1.3s

Using right variant = 10-40x faster!

Best Practices

1. Use Appropriate Variant

```
// Blog post list - use small  
<img src={`\api/images/${id}?variant=small`} />  
  
// Full post view - use medium or original
```

```
<img src={`\api/images/${id}?variant=medium`} />

// Avatar/thumbnail - use thumbnail
<img src={`\api/images/${id}?variant=thumbnail`} />
```

2. Implement WebP with Fallback

```
<picture>
  <source srcset="...?variant=webp" type="image/webp">
  
</picture>
```

3. Lazy Load Images

```

```

4. Specify Dimensions

```

```

Configuration

Adjust Quality Settings

In `image_processor.rs`:

```
// JPEG quality (0-100)
let original_data = Self::to_jpeg(&img, 85)?; // Change from 85

// WebP quality (0.0-100.0)
let webp_data = Self::to_webp(&img, 80.0)?; // Change from 80.0
```

Customize Thumbnail Sizes

```
// Current sizes
Thumbnail => Some(150),
Small => Some(400),
Medium => Some(800),

// Adjust as needed
Thumbnail => Some(200), // Larger thumbnails
```

```
Small => Some(600),      // Larger small
Medium => Some(1200),    // Larger medium
```

Add New Variant

```
// In ImageVariant enum
pub enum ImageVariant {
    Original,
    Thumbnail,
    Small,
    Medium,
    Large,      // NEW: Add this
    WebP,
}

// In process_image function
if width > 1200 || height > 1200 {
    let large = Self::create_thumbnail(&img, 1200)?;
    // ... save large variant
}
```

Storage Impact

Database Usage

Per image (1920x1080 photo):

Variant	Size
Original	320 KB
Thumbnail	8 KB
Small	45 KB
Medium	120 KB
WebP	120 KB
Total	~613 KB

With 100 images: ~61 MB total

Turso Free Tier: 9 GB

Capacity: ~14,600 images

Optimization Tips

1. Aggressive WebP quality:

```
Self::to_webp(&img, 70.0)? // 70% instead of 80%
```

2. Skip medium variant:

```
// Only create if really needed
if width > 1200 || height > 1200 {
    // Create medium
}
```

3. Lower JPEG quality:

```
Self::to_jpeg(&img, 80)? // 80% instead of 85%
```

Testing

Test Image Processing

```
# Upload test image
curl -X POST http://localhost:3000/api/upload/image \
-H "Content-Type: application/json" \
-d '{
  "filename": "test.jpg",
  "content_type": "image/jpeg",
  "data": "..base64.."
}'
```

Verify Variants

```
# Get original
curl -I http://localhost:3000/api/images/{id}

# Get thumbnail
curl -I http://localhost:3000/api/images/{id}?variant=thumbnail

# Compare sizes
curl -I http://localhost:3000/api/images/{id}?variant=webp
```

Check Database

```
-- See all variants for an image
SELECT variant, width, height, size, content_type
FROM images
WHERE parent_id = 'original-image-id';

-- Total storage per image
SELECT
  parent_id,
  COUNT(*) as variant_count,
  SUM(size) as total_bytes,
```



```
SUM(size) / 1024 as total_kb
FROM images
WHERE parent_id IS NOT NULL
GROUP BY parent_id;
```

Troubleshooting

“Failed to process image”

Check: 1. Valid image format (JPEG, PNG, GIF, WebP) 2. Not corrupted 3. Readable by image library

Solution:

```
# Test with simple image
convert -size 100x100 xc:blue test.jpg
base64 test.jpg
# Use in upload request
```

Variants Not Created

Check logs for:

Warning: Failed to save thumbnail variant: ...

Common causes: - Database connection issue - Storage limit reached - Invalid image data

WebP Not Loading

Browser support: - Chrome/Edge: Full support - Firefox: Full support
- Safari: iOS 14+, macOS 11+ - IE: Not supported

Solution: Always provide JPEG fallback

Monitoring

Track Optimization

```
-- Average file size reduction
SELECT
    AVG(CAST(original.size - webp.size AS FLOAT) / original.size * 100) as avg_reduction
FROM images original
JOIN images webp ON webp.parent_id = original.id
WHERE original.variant = 'original'
    AND webp.variant = 'webp';
```

```

-- Most optimized images
SELECT
    original.filename,
    original.size as original_size,
    webp.size as webp_size,
    CAST((original.size - webp.size) AS FLOAT) / original.size * 100 as reduction_pct
FROM images original
JOIN images webp ON webp.parent_id = original.id
WHERE original.variant = 'original'
    AND webp.variant = 'webp'
ORDER BY reduction_pct DESC
LIMIT 10;

```

Storage Usage

```

-- Total storage by variant
SELECT
    variant,
    COUNT(*) as count,
    SUM(size) / 1024 / 1024 as total_mb
FROM images
GROUP BY variant;

```

Migration Guide

From Old System

If you have existing images without variants:

```

# Run migration script
cargo run --bin migrate-variants

```

Migration script:

```

// Fetch all original images without variants
// For each image:
//   - Load from database
//   - Process through ImageProcessor
//   - Save new variants
//   - Link to original

```

Future Enhancements

Planned features:

- ☐ AVIF support (even better compression)

- ☐ Blurhash placeholders
 - ☐ Automatic face detection/cropping
 - ☐ Smart cropping for thumbnails
 - ☐ Progressive JPEG encoding
 - ☐ EXIF data preservation (optional)
 - ☐ Batch re-processing
 - ☐ Custom quality per variant
-

Example: Blog Post Image

```
<!-- Full implementation -->
<article class="blog-post">
  <picture>
    <!-- WebP for modern browsers -->
    <source
      srcset="/api/images/abc123?variant=webp"
      type="image/webp"
    />

    <!-- Responsive sizes -->
    <source
      media="(max-width: 640px)"
      srcset="/api/images/abc123?variant=small"
    />
    <source
      media="(max-width: 1024px)"
      srcset="/api/images/abc123?variant=medium"
    />

    <!-- Default fallback -->
    
  </picture>
</article>
```

Result: - Mobile (400px): Loads 45 KB small variant - Tablet (800px): Loads 120 KB medium variant - Desktop: Loads 120 KB WebP or 320 KB JPEG - Perfect quality at minimal bandwidth!

Summary

What you get: - Automatic thumbnail generation (3 sizes) - WebP conversion (30-80% smaller) - Smart format selection - All variants stored in Turso
- Responsive image URLs - Optimization statistics

Benefits: - 10-40x faster page loads - 85-99% storage savings - Perfect mobile experience - Lower bandwidth costs - Better SEO scores

Zero configuration required - it just works!

Need help? Check the troubleshooting section or review the source code in `image_processor.rs`!